



成都东软学院

实 验 报 告

课程名称： 人工智能提示工程

指导教师： 谢心

学 院： 数智应用技术学院

年级专业： 24 级软件工程

班 级： 24402

学 号： 24068240218

姓 名： 杨陆续

实验（1）

【实验目的】

1. 学习并实践使用 AI 编程助手（如 TRAE/Cursor、vibe coding）进行项目开发。
2. 开发一个能够调用本地 LLM（通过 OpenAI 兼容 API）的最小化 Python 程序。
3. 掌握项目环境配置、敏感信息管理（.env 文件）和基础性能监控（token 消耗、响应速度）的方法。

【实验内容】

1. 使用 AI 编程助手引导，完成项目初始化、环境配置和代码开发。
2. 编写一个 Python 脚本，该脚本能够读取配置文件，调用本地 LLM 的 API，并统计调用过程中的关键指标。

1.2 实验过程：

1. 项目初始化与环境配置：

- 手动创建了一个不包含中文路径的空白项目目录。
- 使用 AI 编程助手（基于“vibe coding”模式），通过提示词引导助手完成以下工作：
 - ① 检查 Python 环境（确保 ≥ 3.12 ）。
 - ② 初始化项目虚拟环境（venv）。
 - ③ 生成.gitignore 文件。
 - ④ 在项目根目录创建 env.example 文件，内容模板包含

OPENAI_API_BASE（即 LM Studio 提供的本地 API 地址，如 `http://localhost:1234/v1`）、MODEL_NAME、OPENAI_API_KEY（可为空或任意值）等字段。

➤ 根据 `env.example` 手动创建并填写了实际的 `.env` 文件。

2. 功能代码开发：

➤ 在 `practice01` 目录下创建 Python 文件（如 `call_llm.py`）。

➤ 编写代码实现以下功能：

① 使用 `python-dotenv` 或类似库读取 `.env` 文件中的配置。

② 使用 `requests` 库（Python 标准 HTTP 库）构造请求，访问由 OPENAI_API_BASE 和 MODEL_NAME 指定的本地 LLM API（Chat Completion 端点）。

③ 发送一个简单的测试提示词（如“请用一句话介绍你自己”）。

④ 从 API 响应中提取回答内容、使用的总 token 数（`usage.total_tokens`）和请求处理时间。

⑤ 计算并打印出响应内容、总耗时、总 token 消耗以及每秒处理的 token 数（tokens/s）。



1.2 实验结果:

1. 成功创建了一个结构清晰、配置独立的 Python 项目。
2. call_llm.py 脚本运行成功，能够正确连接到 LM Studio 提供的本地 API 并获取模型的响应。

① 程序正确输出了模型的回复，并统计出了类似如下的性

能指标:

- ② 回复内容: “你好,我是一个大型语言模型...”
- ③ 总耗时: 2.45 秒
- ④ Token 消耗: 输入 28, 输出 15, 总计 43
- ⑤ 速度: 约 17.55 tokens/s

3. 通过修改.env 文件中的配置,可以灵活切换不同的本地模型或 API 端点。

```
=== Statistics ===  
Prompt tokens: 16  
Completion tokens: 97  
Total tokens: 113  
Time elapsed: 2.28 seconds  
Token speed: 49.63 tokens/second
```

【实验总结】

本次实验成功利用 AI 辅助编程完成了从零开始的本地 LLM 调用项目搭建。掌握了通过 OpenAI 兼容协议与本地模型服务交互的标准方法,并实现了基本的调用监控功能。env.example 与 .env 的分离使用,规范了敏感信息的管理。开发的代码模块为后续的性能调优实验提供了可重复使用和测量的基础工具。